Ossie Hub Instruction

Move a Power BI / Microsoft Fabric model, its data and its business meaning into Databricks, step by step

Every step is shown two ways: the **code way** (commands you type) and the **prompt way** (a sentence you give Claude Code). Each step starts with an explanation in plain words, and the last part explains how our code works.

Part	What it covers
1 The big picture	What we are building, in everyday words, and the vocabulary
2 Getting ready	Install, test, how to read this document
3 The steps	Connect Fabric and Databricks, convert, transfer, ontology, Genie, check
4 How our code works	Folders, the pieces, how a measure is translated, safety, tests
5 Problems and limits	What the messages mean, what is not done yet
6 Quick reference	Every command on one page

1. The big picture

The problem. A company has a Power BI report. Behind it is a model: tables of data, the links between them, and calculations such as "sales this year". That model lives in Power BI or Microsoft Fabric. Now the team wants the same data, the same calculations and the same business meaning available in **Databricks**, without rebuilding everything by hand.

What Ossie Hub does. It reads the model, writes it in a neutral file format called **Ossie** (a YAML text file), writes a second file called the **ontology** that holds the business meaning, and then builds the Databricks version and copies the data across. After copying it checks its own work by recalculating numbers and comparing them.

Layer	Everyday picture	What it is for
Data	The rows in the filing cabinet	Orders, customers, products, locations
Semantic model	The instructions for reading the cabinet	Which tables link together, and how to calculate Sales, Profit, Quantity
Ontology	The dictionary of business words	"A Sales Transaction is placed by a Customer" - what things mean and how they relate

The route the data takes

1. Source	2. Neutral files (Ossie)	3. Databricks
Power BI file (.pbix), or a model in Fabric	Ossie model + ontology + measure report	Tables + data + Metric View + ontology tables, then Genie

Words you will meet

Word	Plain meaning
Semantic model	The tables, links and calculations behind a report.
Ossie	A neutral text format to describe a semantic model so it can move between platforms. Think of it as a PDF for models.
Ontology	A file of business concepts (Customer, Product ...) and how they relate. It sits on top of the semantic model.
Fact table / dimension	The fact table holds the events (one row per sale). Dimensions describe them (who, what, where).
Measure	A named calculation, e.g. Total Sales. In Power BI it is written in DAX; in Databricks in SQL.
DAX	Power BI's formula language. The tool translates it to SQL where it is safe to do so.
Metric View	A Databricks object that holds measures and dimensions together, so the same calculation is used everywhere.
Lakehouse	Fabric's storage for tables (like a folder of data tables).
Catalog and schema	Databricks folders for tables: catalog > schema > table. We use catalog `workspace`.
Genie	Databricks' feature to ask questions about data in plain English.
Dry run	Showing what would happen without doing it. This is the default; adding `--execute` makes it real.
Connection	A saved login to Fabric or Databricks. Secrets are kept in the Windows credential store, never in a file.

2. Getting ready

How to read each step

- **In plain words** explains why the step exists.
- **Code way** is what you type in the terminal. Open a terminal in the project folder first.
- **Prompt way** is a sentence you paste to Claude Code, which runs the commands and reports back.
- **What you should see** shows real output from our own run, so you can compare.

CAREFUL
Every command that writes to Databricks or Fabric is a **dry run** until you add `--execute` at the very end of the same command. Run it once without, read the result, then run it again with `--execute`.

What you need

Need	Detail
Python 3.11 or newer	Check with `python --version`
The project folder	Open a terminal at its root (the folder that contains `ossie_hub`)
Fabric	A workspace you can read, with a lakehouse, and a Microsoft account that can sign in
Databricks	A workspace, a SQL warehouse (serverless is fine) and a personal access token

Install and test (once)

In plain words: the first command downloads the libraries the tool needs. The second runs the tool's own self-checks; they work offline and finish in seconds. `OK` at the end means the install is healthy.

CODE WAY - type this in the terminal
```
python -m pip install -r ossie_hub/requirements.txt
```

CODE WAY - type this in the terminal
```
python -m unittest ossie_hub.tests.test_dax ossie_hub.tests.test_pbix_files
ossie_hub.tests.test_fabric ossie_hub.tests.test_targets ossie_hub.tests.test_connections
ossie_hub.tests.test_ontology_push ossie_hub.tests.test_document
```

WHAT YOU SHOULD SEE
The tests end with `Ran 68 tests ... OK`. It is one long command; the terminal may wrap it over several lines.

PROMPT WAY - paste this to Claude Code
Install the packages from `ossie_hub/requirements.txt` and run the project's unit tests. Tell me if anything fails.

3. The steps

Follow the steps in order. Steps 1 and 2 are done once per computer. Steps 4 to 7 are repeated for each report.

Step	What you do	Result
1	Connect Microsoft Fabric	Saved connection `fab-main`
2	Connect Databricks	Saved connection `dbx-main`
3	Look around Fabric	You know which workspace, model and tables exist
4	Put the Power BI file in `import_models`	File ready
5	Convert the report	Ossie model + ontology + measure report
6	Move the model and data to Databricks	Tables, data, Metric View, verified numbers
7	Send the ontology to Databricks	Four ontology tables, comments and tags
8	Connect Genie	A space where you ask questions
9	Check the result	Queries that prove it

Step 1

Connect Microsoft Fabric

IN PLAIN WORDS

The tool must be allowed to read your Fabric workspace. You sign in once in a browser window, like logging into any Microsoft site. Your password is never stored; the tool remembers only that you are signed in.

CODE WAY - type this in the terminal

```
python -m ossie_hub connect add fabric --name fab-main --set auth_mode=interactive_browser
python -m ossie_hub connect test fab-main
```

- `connect add` asks one question: **Your Microsoft account email**. Type your work or school email. It only pre-selects the account in the sign-in window.
- `connect test` opens the browser sign-in. Approve it. It prints `"ok": true` and the number of workspaces you can see.

GOOD TO KNOW

Your manager does exactly the same on their own computer, with their own email. Nothing is shared. A Tenant ID is only for special cases; add `--advanced` to be asked for it. It is a code or domain, never an email.

CAREFUL

Giving the folder to someone else? A connection is personal. Before handing the folder over, delete `ossie_hub/connectors/fabric/connections/*.json`, `ossie_hub/connectors/databricks/connections/*.json` and the `.ossie_hub` folder. They hold your email, your Databricks host and your sign-in record. Without them, the other person is asked for **their own** email and token when they run `connect add`. (Sharing through Git is already safe: these files are git-ignored.)

PROMPT WAY - paste this to Claude Code

Create a saved Fabric connection named `fab-main` using interactive browser sign-in, then test it and tell me how many workspaces I can see.

Step 2

Connect Databricks

IN PLAIN WORDS

Same idea for Databricks. You give the tool the address of your workspace, which SQL warehouse to use, and a personal access token (a long password you generate in Databricks). The token is stored in the Windows credential store.

Item	Where to find it
Workspace host	The web address of your workspace, e.g. `dbc-xxxx.cloud.databricks.com` (without `https://`)
SQL warehouse	SQL Warehouses page: its name, e.g. Serverless Starter Warehouse
Access token	Profile > Settings > Developer > Access tokens > Generate new token (starts with `dapi`)

CODE WAY - type this in the terminal

```
python -m ossie_hub connect add databricks --name dbx-main --set host=<your-host> --set "
warehouse=<warehouse name>"
python -m ossie_hub connect test dbx-main
```

It asks for the token at a hidden prompt (nothing shows as you type). Paste it there.

CAREFUL

Never paste a token into a chat or a file. If one was exposed, revoke it in Databricks and run `connect add` again with a new one.

PROMPT WAY - paste this to Claude Code

Create a saved Databricks connection named `dbx-main`. Host: <your-host>, SQL warehouse: <name>. I will type the access token myself at the hidden prompt, do not ask me for it in chat. Then test it.

Managing connections

CODE WAY - type this in the terminal

```
python -m ossie_hub connect list
python -m ossie_hub connect test <name>
python -m ossie_hub connect remove <name>
```

Step 3

Look around Fabric

IN PLAIN WORDS

Before converting, you want to know what exists: which workspaces, which models, which lakehouse and which data tables. Each command narrows the previous one down.

CODE WAY - type this in the terminal

```
python -m ossie_hub options fabric workspace --connection fab-main
python -m ossie_hub options fabric semantic_model --connection fab-main --pick
workspace=OssieApache_Workspace
python -m ossie_hub options fabric lakehouse --connection fab-main --pick
workspace=OssieApache_Workspace
python -m ossie_hub options fabric lakehouse_table --connection fab-main --pick
workspace=OssieApache_Workspace --pick lakehouse=Lakehouse_1
```

Command	Lists	Example answer
workspace	Workspaces you can open	OssieApache_Workspace
semantic_model	Models in that workspace	Sales
lakehouse	Lakehouses in it	Lakehouse_1
lakehouse_table	The data tables	Schema2.orders, Schema2.customers, Schema2.products, Schema2.location

PROMPT WAY - paste this to Claude Code

Using my saved Fabric connection `fab-main`, show me the workspaces I can see. For `OssieApache_Workspace` list its semantic models, lakehouses, and the tables inside `Lakehouse_1`.

Step 4

Put the Power BI file in place

IN PLAIN WORDS

The tool reads the report from the folder `import_models`. Copy your `.pbix` file there. There are two kinds of report: one that carries its own data inside, and a small "live connection" file that only points to a model in Fabric. The tool recognises which one it is.

CODE WAY - type this in the terminal

```
dir import_models
```

You should see your file in the list, e.g. `Sales_Dashboard.pbix` (about 700 KB, carries its data) or `Sales_Dashboard_fabric.pbix` (about 8 KB, live connection to Fabric).

PROMPT WAY - paste this to Claude Code

Check that `import_models/Sales_Dashboard_fabric.pbix` exists and tell me its size.

Step 5

Convert the report: Ossie model, ontology and measure report

IN PLAIN WORDS

One command reads the report and writes three files into a folder named after the report. The Ossie model is the neutral copy of the model. The ontology is the business dictionary. The measure report lists every calculation and whether it could be translated safely. Both YAML files are checked against the official schema automatically.

A report that carries its own data

CODE WAY - type this in the terminal

```
python -m ossie_hub pbix import_models/Sales_Dashboard.pbix
```

A report connected to Fabric (the data stays in Fabric)

CODE WAY - type this in the terminal

```
python -m ossie_hub pbix import_models/Sales_Dashboard_fabric.pbix --connection fab-main
```

What you get

RESULT

```
output_models/
  Sales_Dashboard_fabric/
Sales_Dashboard_fabric_ossie.yaml <- the Ossie model
Sales_Dashboard_fabric_ontology.yaml <- the ontology
Sales_Dashboard_fabric_translation_report.md <- which measures became SQL
data/ <- CSV files, only for reports that carry data
```

WHAT YOU SHOULD SEE

Report : Sales_Dashboard_fabric.pbix (live)

Data source : Fabric ... model 'Sales' in workspace 'OssieApache_Workspace'

Ossie model : ... validation: PASSED

Ontology : ... validation: PASSED (4 entity concepts: SalesTransaction, Customer, Product, Location)

Measures : 6 of 6 translated to SQL

For `Sales_Dashboard.pbix` (embedded data) the result is **23 of 31** measures translated. The other 8 compare each group with the others in a visual (`ALLSELECTED`); that cannot be written as a model-level calculation, so they are kept as the original DAX with the reason written in the report. **They are never guessed.**

PROMPT WAY - paste this to Claude Code

Convert `import_models/Sales_Dashboard_fabric.pbix` with `python -m ossie_hub pbix`, using my saved Fabric connection `fab-main`. It should create the folder `output_models/Sales_Dashboard_fabric/` with the Ossie model, the ontology and the measure report, and validate both. Then summarise the tables, relationships, ontology concepts, and which measures were translated and why any were not.

What the ontology contains for this model

Relationship	Reads as
SalesTransaction.placed_by	A Sales Transaction is placed by a Customer

Relationship	Reads as
SalesTransaction.for_product	A Sales Transaction is for a Product
SalesTransaction.delivered_to	A Sales Transaction is delivered to a Location

Step 6

Move the model and data to Databricks

IN PLAIN WORDS

This is the conversion into Databricks format. The tool takes the Ossie model and builds the Databricks version: a schema (a folder), one table per dataset, and a Metric View that holds the translated calculations. It then reads the rows from the Fabric lakehouse (or from the CSV files) and loads them. Finally it recalculates some numbers independently and compares them with what Databricks returns.

Report connected to Fabric: data read from the lakehouse

DRY RUN - changes nothing

```
python -m ossie_hub migrate --to databricks --source fabric --connection-fabric fab-main
--from-yaml output_models/Sales_Dashboard_fabric/Sales_Dashboard_fabric_ossie.yaml --catalog
workspace --schema sales_dashboard_fabric --connection dbx-main
```

REAL RUN - the same command with --execute at the end

```
python -m ossie_hub migrate --to databricks --source fabric --connection-fabric fab-main
--from-yaml output_models/Sales_Dashboard_fabric/Sales_Dashboard_fabric_ossie.yaml --catalog
workspace --schema sales_dashboard_fabric --connection dbx-main --execute
```

Report with its own data: rows read from the CSV files

CODE WAY - type this in the terminal

```
python -m ossie_hub migrate --to databricks --source csv --csv-dir
output_models/Sales_Dashboard/data --from-yaml output_models/Sales_Dashboard/Sales_Dashboard_ossie.y
aml --fact orders --catalog workspace --schema sales_dashboard --connection dbx-main --execute
```

Why a new schema? Everything for one model goes into its own schema: data tables, Metric View and later the ontology tables. Two models in one schema would overwrite each other.

WHAT YOU SHOULD SEE

The end of a real run lists the verification, one line per measure:

OK total_quantity: expected 37,873.00 got 37,873.00

OK order_count: expected 5,009.00 got 5,009.00

OK profit_margin: expected 0.12 got 0.12

OK avg_discount: expected 0.16 got 0.16

Any BAD line means the transfer cannot be trusted.

Repeated keys in the lakehouse (for example the same product ID twice) would inflate totals when tables are joined. The tool keeps the first row, records the different variants in an `Alternate_Records` column, and reports what it collapsed.

PROMPT WAY - paste this to Claude Code

Do a **dry run** of moving `output_models/Sales_Dashboard_fabric/Sales_Dashboard_fabric_ossie.yaml` to Databricks, reading the rows from the Fabric lakehouse it points to (saved connections `fab-main` and `dbx-main`), into the new schema `sales_dashboard_fabric` in catalog `workspace`. Show me which Fabric data it will read, the row counts and the plan. Do not execute until I say so.
Then: Execute it with `--execute` and show me the verification table.

Step 7

Send the ontology to Databricks

IN PLAIN WORDS

So far Databricks has the data and the calculations. The ontology adds the meaning. The tool writes four small tables into the same schema, and it also writes business names as comments and tags on the data tables and columns, where people and Genie can see them.

DRY RUN - changes nothing

```
python -m ossie_hub ontology push --from-yaml output_models/Sales_Dashboard_fabric/Sales_Dashboard_fabric_ontology.yaml --catalog workspace --schema sales_dashboard_fabric --connection dbx-main
```

REAL RUN - the same command with --execute at the end

```
python -m ossie_hub ontology push --from-yaml output_models/Sales_Dashboard_fabric/Sales_Dashboard_fabric_ontology.yaml --catalog workspace --schema sales_dashboard_fabric --connection dbx-main --execute
```

Created in Databricks	What it holds
ontology_info	which ontology, which model, when pushed
ontology_concepts	each business concept: name, type, key, source table
ontology_relationships	each relationship with its sentence and multiplicity
ontology_mappings	which table and column each concept comes from
comments and tags	business name on every data table, and a comment on every mapped column

WHAT YOU SHOULD SEE

ontology_info 1 row | ontology_concepts 25 | ontology_relationships 24 | ontology_mappings 24 | annotations 32 statements. For a real run it also prints `loaded N rows` per table.

Safety built in

- Nothing is written without `--execute`.
- Before sending, the ontology is checked for completeness: every entity needs an identifier and a table mapping. If not, it stops and names the entity.
- The four tables carry a marker in their comment. Re-running refreshes them. A table with the same name that the tool did not create is never touched unless you add `--overwrite`.
- Data tables are only touched for comments and tags. `--no-annotate` skips even that.

PROMPT WAY - paste this to Claude Code

Do a **dry run** of pushing `output_models/Sales_Dashboard_fabric/Sales_Dashboard_fabric_ontology.yaml` to Databricks (connection `dbx-main`) into catalog `workspace`, schema `sales_dashboard_fabric`. Tell me the tables and row counts it would write. Do not execute until I say so.
Then: Execute it with `--execute`, then list every dimension related to the fact table and show me the result.

Step 8

Connect Genie

IN PLAIN WORDS
Genie lets people ask the data questions in plain English. It answers from the tables, Metric Views and comments attached to a Genie space. This step is done in the Databricks screen.

- Open **Genie** and choose **New**. The *Connect your data* window opens.
- The **For you** tab shows only recently used tables. Click **All** (or search `sales_dashboard_fabric`).
- Tick `orders`, `customers`, `products`, `location` and the four `ontology_*` tables.
- Open the **Metric views** tab and tick `sales_dashboard_fabric_metrics` (a Metric View is not listed with tables).
- Check that **Selected** shows 9 items and click **Create**.

Suggested instruction text for the space (adapt to your model)

PASTE INTO THE SPACE INSTRUCTIONS
`orders` is the fact table: one row per sales line. It links to `customers` by Customer_ID, to `products` by Product_ID and to `location` by Postal_Code. The tables `ontology_concepts`, `ontology_relationships` and `ontology_mappings` describe the business terms and how they relate: use them to answer "what is X" and "how is X related to Y". Use the metric view for totals and averages.

Questions to try

- What is a Sales Transaction related to?
- Total sales by customer segment.
- Which locations receive the most sales transactions?

CAREFUL
The content of the schema was checked in Databricks. How well Genie answers a given business word was **not** measured yet. Try the questions and keep the good answers as examples in the space.

PROMPT WAY - paste this to Claude Code
I want to connect schema `workspace.sales_dashboard_fabric` to a Genie space. List exactly which tables and which Metric View to add, and write the instruction text for the space from the ontology.

Step 9

Check the result in Databricks

IN PLAIN WORDS

Prove that everything arrived: tables exist, rows are there, measures return numbers, and the ontology tables answer questions. These are run in the Databricks SQL editor.

SQL - run in the Databricks SQL editor

```
SHOW TABLES IN workspace.sales_dashboard_fabric;
SELECT COUNT(*) FROM workspace.sales_dashboard_fabric.orders;
SELECT concept, relationship, related_concept, verbalization
FROM workspace.sales_dashboard_fabric.ontology_relationships
WHERE relationship_kind = 'link';
```

WHAT YOU SHOULD SEE

Row counts in our run: orders 9,994 | customers 793 | products 1,862 | location 631.

The last query returns three rows: placed_by, for_product, delivered_to.

A Metric View is queried with `MEASURE(<name>)`, for example `SELECT <dimension>, MEASURE(<measure>) FROM <metric view> GROUP BY <dimension>`.

PROMPT WAY - paste this to Claude Code

In schema `workspace.sales_dashboard_fabric` show me the tables with their row counts, the Metric View, and the table and column comments. Confirm that the ontology tables have rows and that each of the four data tables has an `ontology_concept` tag.

Optional: go straight from a Fabric model (no .pbix)

If you have no report file, convert the model directly from Fabric and transfer it with one command.

CODE WAY - type this in the terminal

```
python -m ossie_hub read-model fabric --connection fab-main --pick workspace=OssieApache_Workspace
--pick semantic_model=Sales -o output_models/live_sales_ossie.yaml
python -m ossie_hub transfer --from fab-main --to dbx-main --workspace OssieApache_Workspace
--model Sales --catalog workspace --schema sales_from_fabric
```

4. How our code works

This part explains what is inside the project, so that you can describe it to someone else or find the right place when something needs changing.

The folders

Folder / file	What it is
`import_models/`	Your source Power BI files
`output_models/`	Everything the tool generates, one folder per report
`ossie_hub/`	The application. Run it with `python -m ossie_hub <command>`
`validation/validate.py`	Official checker for Ossie files (add `--schema ontology/ontology.json` for ontologies)
`tools/`	Small helper scripts, e.g. `ossie_to_ontology.py`
`docs/`	This instruction and the written guide `ossie_hub_guide.md`
`Knowledge/`	Reference material and earlier hand-made models

The pieces inside ossie_hub

File	Job	In everyday words
`__main__.py`	The command line: types, connect, options, read-model, pbix, migrate, transfer, ontology, publish	The front desk: reads your command and calls the right worker
`connections.py, spec.py`	Saved connections and the questions asked when adding one	Remembers logins; secrets go to the Windows credential store
`connectors/fabric`	Talks to Fabric: workspaces, models, lakehouse tables, sign-in	Our messenger to Fabric
`connectors/databricks`	Runs SQL on a Databricks warehouse; reads models back	Our messenger to Databricks
`pbix.py`	Opens a .pbix; finds tables, relationships and measures; exports data	Reads the Power BI file
`dax.py, tmdl.py`	Translates DAX formulas to SQL by strict rules	The translator that refuses to guess
`outputs.py`	Writes the Ossie model, ontology and report, and validates them	Packs and checks the results
`ontology.py`	Builds the ontology from the Ossie model; checks it is complete	Writes the business dictionary
`targets/planner.py`	Plans the Databricks version: tables, joins, Metric Views, bridges	The architect's drawing
`executor.py, sources.py`	Runs the plan, reads source rows, loads data, verifies measures	The builder and the inspector
`document.py`	Reads old and new Ossie file shapes, writes the current one	Translates between two spellings of the same file
`vendor/ossie_databricks`	Python Metric View converter kept from upstream (upstream replaced it with Java)	A saved copy of a tool we depend on
`ontology_push.py`	Flattens the ontology into four tables and writes comments and tags	Sends the dictionary to Databricks

File	Job	In everyday words
`transfer.py, fabric_writer.py`	Whole-trip transfers, and writing back to Fabric	Fabric <-> Databricks

What happens in one conversion

#	Stage	What happens
1	Read	The report (or the Fabric model) is read: tables, columns, relationships, measures
2	Choose the fact table	The table most of the measures are written against, after excluding pure dimensions
3	Translate	Each DAX measure becomes SQL if the rules fully understand it, otherwise it stays DAX with a reason
4	Write	Ossie model, ontology and measure report are written and checked against the schemas
5	Plan	The Databricks tables, joins and Metric View are planned (dry run stops here)
6	Load	Schema and tables are created, rows are loaded, row counts are checked
7	Verify	Measures are recomputed independently and compared with Databricks
8	Ontology	The four ontology tables, comments and tags are written

How a measure is translated, and why mistakes are avoided

A Power BI measure is DAX; Databricks needs SQL. The translator in `dax.py` follows one rule above all: **a translation is never silently wrong.**

Rule	Meaning
Reject, never guess	Anything not fully understood stays as the original DAX, with the reason in the report
Row-level filters only	A filter that contains an aggregate depends on what the viewer selected, so it is rejected
Viewer-dependent logic rejected	SELECTEDVALUE, ALLSELECTED, RANKX, TOPN, many time functions have no model-level SQL equivalent
Row counts only on the fact table	COUNTROWS on a dimension would count joined rows, so it is rejected
Differences are recorded	Where SQL differs subtly from DAX, the measure's description says so

Example of what is translated: `CALCULATE([Sales], FILTER(ALL(col), YEAR(col) = 2023))` becomes `SUM(Sales) FILTER (WHERE EXTRACT(YEAR FROM Order_Date) = 2023)`. Tested on six real reports: 77 of 112 measures translated; every untranslated one has a stated reason.

How the ontology is built

- One **entity** per dataset (SalesTransaction, Customer, Product, Location), with a business name.
- One **value type** per column (SalesAmount, CustomerName ...).
- One **relationship** from the fact to each dimension, with a sentence in both directions.
- **Mappings** that bind every concept back to a table and column.
- Business names come from two tables at the top of `ontology.py`; unknown tables or columns get a name made from their own names. Review the names before publishing.
- If the model declares no key for a table, the tool uses a column that looks like an identifier, else the first column. Every entity always gets an identifier and a mapping, and the push stops if one does not.

Kept up to date with Apache Ossie

The project was brought up to date with the official Apache Ossie repository (125 newer commits). **No command changed**, so every step in this document works exactly as written.

What upstream changed	What we did
One model per document: the `semantic_model:` list wrapper was removed	Every file we write uses the new form and passes the official validator. Old files are still read. Existing files in `output_models/` and `Knowledge/` were converted in place with identical content
The ontology's embedded model needs a version; embedded mappings are deprecated in favour of a separate mapping file	The ontology embeds a versioned model and validates. It stays one ontology file per report. A separate mapping file is not generated yet
The Python Databricks converter was replaced by a Java one	We keep the last Python version inside the project (`ossie_hub/vendor`), so the Metric View output is identical
A new official Microsoft converter (`ossie-microsoft`): lossless round trip, keeps DAX	Not used by our pipeline. It does a different job: we translate DAX to SQL and move data. Our extension name stays `POWERBI` so the two never read each other's data

The safety rules

Rule	How it works
Dry run by default	Nothing is written unless `--execute` is given
No silent overwrite	A table that already holds rows stops the run unless `--overwrite` is given
Secrets stay out of files	Tokens live in the Windows credential store or an environment variable
Self-checking	After loading, measures are recomputed from the source rows and compared
Own tables only	The ontology tables carry a marker; foreign tables with the same name are never replaced
Complete or nothing	An incomplete ontology is refused before anything is sent

The tests

Test file	What it protects
`test_dax`	Translation rules, including what must be rejected, and the fact-table choice
`test_pbix_files`	Telling embedded reports from Fabric-connected ones
`test_fabric`	The Fabric connector against a fake Fabric
`test_targets`	The Databricks planner: renames, bridges, duplicate keys
`test_connections`	Secrets never written to a file; sign-in validation
`test_document`	Files are written in the current one-model-per-document form; old and new forms are both read; the Databricks plan is identical from either
`test_ontology_push`	Ontology flattening; refusal of an incomplete or foreign table; a model with none of the sales vocabulary

5. Problems and limits

Messages and what they mean

You see	Meaning and fix
`refusing to continue: ... already contain rows`	The schema already has data. Use a new schema name, or add `--overwrite` if replacing is intended
`the ontology is incomplete, nothing was sent`	An entity has no identifier or mapping. The message names it. Regenerate the ontology (step 5); nothing was written
Authentication failed (Databricks)	Token revoked or expired. Create a new one and run `connect add databricks` again
Fabric sign-in picks the wrong organisation	Add the Tenant ID with `connect add fabric ... --advanced`
`no saved connection 'x'`	Run `python -m ossie_hub connect list` and use an existing name
Ontology table exists but a data table has no comment	The data was not in the schema when the ontology was pushed. Do step 6, then step 7 again
Genie shows only 5 tables	The For you tab lists recent items. Use All and Metric views
A .pbix gives no rows	It is a live connection with no embedded data. Use step 5 with `--connection fab-main`
Names with spaces	Wrap in double quotes, e.g. `--set "warehouse=Serverless Starter Warehouse"`

What is not done yet

- **Measure translation is partial by design**: 23 of 31 for the Sales dashboard; the rest are kept as DAX.
- **Business synonyms** are not written into the Metric View yet, so Genie may not match a word like "client" until they are.
- **Genie answers** were not measured; the space content was only checked in Databricks.
- **Business names** for a brand-new model are derived from table and column names; review them.
- **Composite keys**: a relationship on several columns links through the first column only.
- **Data movement** goes in batches through SQL: fine for tens of thousands of rows, not for millions.
- **The year is fixed** at 2023 in the Sales dashboard because the report's parameter is a constant.
- **Databricks converter** is a vendored copy of the last Python version; if upstream's Java converter changes behaviour, ours will not follow automatically.
- **The browser interface** exists in the code but is not ready, so this instruction uses the terminal only.

6. Quick reference

Task	Command
Install	`python -m pip install -r ossie_hub/requirements.txt`
Connect Fabric	`python -m ossie_hub connect add fabric --name fab-main --set auth_mode=interactive_browser`
Connect Databricks	`python -m ossie_hub connect add databricks --name dbx-main --set host=<host> --set "warehouse=<wh>"`
List / test connections	`python -m ossie_hub connect list | python -m ossie_hub connect test <name>`
Report with own data	`python -m ossie_hub pbix <file.pbix>`
Report connected to Fabric	`python -m ossie_hub pbix <file.pbix> --connection fab-main`
Fabric model directly	`python -m ossie_hub read-model fabric --connection fab-main --pick workspace=<ws> --pick semantic_model=<model> -o <file_ossie.yaml>`
Validate model	`python validation/validate.py <file.yaml>`
Validate ontology	`python validation/validate.py <file.yaml> --schema ontology/ontology.json`
Transfer (Fabric rows)	`python -m ossie_hub migrate --to databricks --source fabric --connection-fabric fab-main --from-yaml <ossie.yaml> --catalog <c> --schema <s> --connection dbx-main`
Transfer (CSV rows)	`python -m ossie_hub migrate --to databricks --source csv --csv-dir <dir> --from-yaml <ossie.yaml> --fact <fact> --catalog <c> --schema <s> --connection dbx-main`
Push ontology	`python -m ossie_hub ontology push --from-yaml <ontology.yaml> --catalog <c> --schema <s> --connection dbx-main`
Make any of them real	`add --execute at the very end of the same command`
Run the tests	`python -m unittest ossie_hub.tests.test_dax ossie_hub.tests.test_pbix_files ossie_hub.tests.test_fabric ossie_hub.tests.test_targets ossie_hub.tests.test_connections ossie_hub.tests.test_ontology_push ossie_hub.tests.test_document`